Java Threads, Thread Pools, Concurrency, and Reactor Schedulers

A beginner-friendly guide to understanding `Schedulers.boundedElastic()` in a Spring Boot + Spring WebFlux + Spring AI application.

Table of Contents

1. What is a Thread?
2. Multiple Threads
3. Blocking Operations
4. Thread Pool
5. Concurrency vs Parallelism
6. Asynchronous Execution
7. Reactor and Reactive Programming
8. Reactor Scheduler
9. Common Reactor Schedulers
10. `Schedulers.boundedElastic()`
11. Shared Scheduler Concept
12. What Happens with 1000 Requests?
13. `schedule()` Method
14. Apply It to the Lovable Clone
15. Explain the Exact Code
16. Important Distinctions
17. Common Misconceptions
18. Practical Rules to Remember

---

1. What is a Thread?

What a thread is, in simple terms

A **thread** is a single, independent **path of execution** through your code.

Think of your program as a recipe. A thread is the cook who reads the recipe line by line and performs each instruction.

```
Program = a book of instructions
Thread  = one person walking through the instructions, one line at a
time
```

When you write `System.out.println("Hello")`, a thread executes that line. Every single line of Java code you have ever written was executed by **some thread**.

Why Java programs need threads

A program usually needs to do more than one thing at the same time:

- Listen for incoming HTTP requests
- Read a file
- Query a database
- Call an external API

Without threads, your program could only do one of those at a time. While a database call was in progress, the program would be **frozen** and unable to handle anything else.

How a thread executes Java code

When you call a method:

```
public void greet() {
    String message = "Hello";
    System.out.println(message);
}
```

A thread:

1. Enters the `greet()` method
2. Reserves space on the **call stack** for the method
3. Executes each line top to bottom
4. Returns, removing the method from the stack

When a program starts, the JVM creates one thread automatically: the **main thread**. That is the thread that runs your `public static void main(String[] args)`.

Simple real-world analogy

A thread is like a **worker in an office**:

- The office = the JVM / process
- The worker = one thread
- A task assigned to the worker = a method or a unit of work
- The worker can only do **one thing at a time**, but an office has **many workers**

Relationship: application, process/JVM, CPU, thread

```
                  CPU (the brain, has several cores)
                        |
                        ▼
            JVM (the Java process, an application running)
               |  |  |  |  |          ← threads
              T1  T2 T3 T4 T5         ← each thread is a path of
execution
               |
               | each thread's instructions
               ▼
         Physical CPU core executes the instructions
```

Key relationships:

- **Application** → a program you run (e.g., your Spring Boot app)
- **Process / JVM** → a running instance of an application; it has its own memory
- **Thread** → a path of execution *inside* a process; threads of the same process share the process's memory
- **CPU** → the hardware that actually runs instructions; a CPU core can genuinely execute one thread's instructions at a time

> **Key idea:** threads are not magic. They are the JVM's way of splitting the work of a process into multiple tracks so that the CPU can juggle them.

A thread is a path of execution

This is the most important sentence of this section:

> **A thread is a path of execution.** It walks through your code, one statement at a time.

---

2. Multiple Threads

Why applications use multiple threads

Because a single thread can only execute one line of code at a time. If your application had one thread, handling two users at once would be impossible — while serving user 1, user 2 would wait.

Multiple threads let your application:

- Serve many HTTP requests "at the same time"
- Keep one slow operation (a file download) from blocking other work
- Use the CPU better

How multiple requests can be handled concurrently

Sequential execution — one after another:

```
Request A ──────────────► Request B ──────────────► Request C
   (finishes)                (finishes)                (finishes)

Time:  A takes 3s, B takes 3s, C takes 3s  →  total = 9s
```

Concurrent execution — overlapping in time:

```
Thread 1: Request A ├──────────┤
Thread 2:             ├──────────┤ Request B
Thread 3:                        ├──────────┤ Request C
```

Even on a single-core CPU, threads are **switched** very quickly between tasks (this is called *time-slicing*), which makes the work appear simultaneous.

Simple Spring Boot web-server example

A Spring Boot application with `spring-boot-starter-web` runs an embedded web server (Tomcat).

- Tomcat owns a **thread pool** of worker threads.
- For each incoming HTTP request, Tomcat **assigns a worker thread** to run your controller method.
- That thread runs the code in your controller (often on the same thread that called your service layer).

```
Browser
  |   GET /api/notes
  ▼
Tomcat (Spring Boot web server)
  |  picks a worker thread from its pool
  ▼
Thread T5 runs:  NotesController.getNotes() → NotesService →
NotesRepository → DB
```

While T5 waits for the database, other threads (T1, T2, ...) are still serving other requests.

Sequential vs concurrent, in one picture

```
SEQUENTIAL (one thread does everything):
  ┌──────────┐   ┌──────────┐   ┌──────────┐
  | request1 |   | request2 |   | request3 |
  └──────────┘   └──────────┘   └──────────┘


CONCURRENT (multiple threads):
  T1 ┌──────────┐
  T2    ┌──────────┐
  T3       ┌──────────┐
```

---

3. Blocking Operations

What a blocking operation means

A **blocking operation** is any operation that **makes the current thread stop and wait** until something finishes, doing nothing useful in the meantime.

Example: when you write `resultSet = statement.executeQuery(...)`, the calling thread stops and waits until the database returns the rows.

Why these operations can take time

The thread is not doing the work itself — it is **waiting on another system**:

- **Database calls** (PostgreSQL): the query has to travel over the network, be planned and executed by the database server, and results travel back.
- **File I/O**: reading/writing a disk is dramatically slower than RAM.
- **MinIO operations**: an HTTP request to a remote object-storage server over the network.
- **External HTTP calls** (e.g., calling OpenRouter): waiting for a completely different server on the internet.

```
Thread
  ↓
MinIO request  ──────────►  network → MinIO server
  ↓
Waiting... (thread does nothing, just waits)
  ↓
Response ◄──────────────── MinIO server replies
  ↓
Continue execution
```

What happens to a thread while it waits

While a thread waits for a blocking operation, it is **occupied but idle**:

- It cannot do any other work.
- It is not using the CPU productively.
- It is just sitting there, blocked.

```
Thread T7 timeline:
  [run code] [run code] [WAIT..... for MinIO ........] [run code] [run
code]
                          └──────── thread is busy but doing nothing
────────┘
```

Why blocking the wrong thread reduces responsiveness

In a Spring Boot web app, the thread serving a request is valuable. If you block it for a slow I/O call:

- That thread can't serve another request.
- If too many requests block for too long, the server's thread pool runs out.
- New requests then wait in a queue → the whole app feels slow and unresponsive.

> **Big idea:** a blocking operation "consumes" a thread for the entire duration of the wait, even though the thread is not doing useful work. That is why blocking the wrong thread is bad.

---

4. Thread Pool

What a thread pool is

A **thread pool** is a fixed set of **reusable worker threads** that are created once and kept alive, ready to execute tasks that are submitted to them.

Why not create a new thread for every small task

Creating a thread is expensive:

- It allocates memory (each thread has its own stack).
- It requires operating-system resources.
- It takes time.

If you created a new thread for every tiny task (say 1000 tasks → 1000 threads), you would:

- Waste memory
- Spend time creating and destroying threads

- Overwhelm the OS scheduler

A thread pool solves this by **reusing** a limited number of threads.

Reusable worker threads

```
Worker threads (already created, kept alive):

  Worker 1 ─┐
  Worker 2 ─┤
  Worker 3 ─┼── waiting for tasks
  Worker 4 ─┤
  Worker 5 ─┘

A task arrives → one free worker picks it up → executes it → becomes
free again
```

How tasks are submitted to a thread pool

You submit work to the pool. The pool hands it to an available worker, or queues it if all workers are busy:

```
ExecutorService pool = Executors.newFixedThreadPool(4);

pool.submit(() -> System.out.println("Hello from a worker"));

pool.shutdown(); // do not submit new tasks; shut down when all tasks
finish
```

Task vs thread — the critical difference

- A **task** is a piece of work (a unit of code you want to run).
- A **thread** is the executor that runs tasks.

One thread can run many tasks over its lifetime — one after another. Tasks are submitted; threads execute them.

```
TASK      = the job to be done   (like a document to be typed)
THREAD    = the person typing     (the executor)
THREAD POOL = a team of typists   (reusable executors)
```

Many tasks, limited threads

This is the single most important idea in this document:

> **Many tasks can exist while the number of threads remains limited.**

```
10 Threads
100 Tasks

10 tasks → executing right now (one per thread)
90 tasks → waiting for an available worker
```

When a worker finishes one task, it picks up the next waiting task.

---

5. Concurrency vs Parallelism

Concurrency

Concurrency is about *structure* — **dealing with many things at once**, making progress on multiple tasks by interleaving them.

- You have multiple tasks.
- They make progress by taking turns (even if only one runs at an exact instant).
- This is like a single cook preparing several dishes: chops the onions for dish 1, then stirs dish 2, then back to dish 1.

```
One core:
  Task A ▓▒▓▒▓▒▓▒▓
  Task B ▒▓▒▓▒▓▒▓▒

Both tasks make progress, but only one runs at any instant.
```

Parallelism

Parallelism is about *execution* — **actually doing many things at the same time**.

- You have multiple CPU cores.
- Each core runs its own thread genuinely simultaneously.

```
Two cores:
  Core 1: Task A ████████
  Core 2: Task B ████████

Both run at the same instant.
```

Multiple threads vs multiple CPU cores

- **Multiple threads** → a design choice in your program. You can have 100 threads on a machine with only 4 cores.
- **Multiple CPU cores** → hardware capability. Each core can genuinely execute one thread's instructions at a time.
- If you have more threads than cores, the OS **time-slices**: it switches which thread runs on each core, very fast.

```
| Concept      | Multiple threads        | Multiple CPU cores          |
|--------------|-------------------------|-----------------------------|
| What it is   | A program design choice | Hardware capability         |
| Allows       | Concurrency (interleaving) | Parallelism (true
simultaneity) |
| Limits       | You, your code          | The physical machine        |
```

Simple rule of thumb: concurrency = structure; parallelism = execution. Threads give you concurrency; cores give you parallelism.

---

6. Asynchronous Execution

Synchronous execution

Synchronous means: you start an operation, **wait** for it to finish, and only then continue to the next line.

```
String result = httpClient.get("https://api.example.com/data"); //
blocks here
System.out.println("Got: " + result); // only runs AFTER the response
arrives
```

The thread waits the entire time the HTTP call is in flight.

Asynchronous execution

Asynchronous means: you *kick off* an operation and **move on immediately**; the result arrives later, and you handle it when it is ready (via a callback, a future, or a reactive stream).

```
// Pseudo-code: start the call, do not wait
Future<String> future =
httpClient.getAsync("https://api.example.com/data");

System.out.println("I did NOT wait. Running other code now...");

String result = future.get(); // waits here, only when you actually need
the value
```

The current thread is free to do other useful work while the slow operation is in progress elsewhere.

Why moving blocking work to another thread helps

The real trick of many asynchronous patterns:

- The **blocking** work (waiting on a DB, MinIO, an HTTP API) is done by a **different thread**.
- The **main flow** (e.g., the reactive pipeline) never blocks.

```
BAD:  the request thread blocks
   [request thread] ── wait for MinIO ──► [frozen, can't serve anyone]

GOOD: the request thread delegates the blocking work
   [request thread] continues immediately  ──► back to serving requests
          │
          ▼
   [worker thread]  ── wait for MinIO ──► returns when done
```

"Another thread" vs "another process" — important distinction

- **Another thread** → same JVM, same memory, same application. Threads can share data. This is what happens inside a single Spring Boot app.
- **Another process** → a completely separate program (a separate JVM, a separate server) with its own memory. Processes cannot directly share memory; they communicate over the network.

In our examples, the blocking work moves to **another thread in the same JVM** — not to another process.

```
Same JVM (one process)
┌──────────────────────────────────────┐
│  main flow thread  ──►  worker thread  │   ← same memory, same app
└──────────────────────────────────────┘


vs.

Process A  ──(network)──►  Process B     ← separate apps, no shared
memory
```

---

7. Reactor and Reactive Programming

Why Reactor exists

Traditional request-per-thread models use one thread per in-flight request, and threads *wait* while doing blocking I/O. Under heavy load that wastes resources.

Reactive programming with Reactor lets an application **handle many I/O-heavy operations with very few threads**, because code is organized around *events* (data arriving) instead of *waiting*.

Mono and Flux

- `Mono<T>` → a stream that emits **0 or 1** item, then completes.
  - Perfect for a single value: a saved entity, one HTTP response body, one user.
  - `Mono.just(42)`, `Mono.empty()`, `Mono.error(new RuntimeException("boom"))`
- `Flux<T>` → a stream that emits **0 to N** items, then completes.
  - Perfect for multiple values: a list of users, chunks of a streamed AI response.
  - `Flux.just("a", "b", "c")`

```
Mono:    ---(value)--->
Flux:    ---(a)---(b)---(c)--->
```

Reactive pipeline

A pipeline is a chain of operations that transform data as it flows:

```
Flux.just("1", "2", "3")
    .map(Integer::parseInt)     // 1, 2, 3
    .filter(n -> n % 2 == 1)    // 1, 3
    .map(n -> n * 10)           // 10, 30
    .subscribe(System.out::println); // prints 10, 30
```

Important: **nothing happens until you subscribe.** Building the pipeline only describes *what* will happen; subscribing makes it actually run.

Non-blocking at a high level

In a truly non-blocking design:

- You never pause a thread waiting for I/O.
- You register interest in data: "when data arrives, call this function."

- The same small set of threads services *thousands* of in-flight operations by reacting to events.

```
Thread serving request A ──→ "data will arrive later" ──→ services
request B meanwhile
        ... data for A arrives ... → resume A
```

How reactive apps handle many I/O-heavy operations efficiently

The same event loop can handle thousands of concurrent slow operations because **no thread is ever idle-waiting**:

```
1 thread services 1000 slow operations:

 op1 ▓▒▒▒▒▒▒▓ (waiting...)  → event arrives → resume
 op2 ▒▓▒▓▒▒▒▒ (waiting...)  → event arrives → resume
 ...
```

Important: `Flux` does NOT automatically mean multiple threads

A `Flux` is just a *description of a stream of values*. It says nothing about threads.

By default, all pipeline operations run **on the thread that subscribes** — sequentially. You need a **Scheduler** (Section 8) to move work to other threads.

```
// These all run on the subscribing thread, one after another:
Flux.range(1, 1000)
    .map(i -> i * 2)     // same thread
    .filter(i -> i > 10) // same thread
    .subscribe(i ->
System.out.println(Thread.currentThread().getName()));
```

---

8. Reactor Scheduler

What a Scheduler is

A **Scheduler** is Reactor's abstraction for **where work executes**. It decides *which thread(s)* a piece of reactive work runs on.

It is Reactor's version of a thread pool — a place that owns threads and hands them out to run tasks.

Why Reactor needs schedulers

By default, reactive code runs on the subscribing thread. To control:

- which thread executes blocking work,
- which threads handle CPU-heavy work,
- how many threads are used for parallel work,

you need a way to choose threads. That is exactly what a Scheduler provides.

How schedulers determine where tasks execute

Three core methods on a `Scheduler`:

Method	What it does
`schedule(Runnable)`	Runs the task asynchronously on one of the scheduler's threads
`schedulePeriodically(...)`	Runs a task repeatedly with a fixed delay
`schedule(Runnable, delay, TimeUnit)`	Runs the task after a delay

Operators like `.subscribeOn()` and `.publishOn()` let you say which scheduler a pipeline (or a section of it) runs on:

```
Flux.range(1, 10)
    .subscribeOn(Schedulers.parallel())  // upstream work runs on
parallel scheduler threads
    ...
```

Difference between a Scheduler and a Thread Pool

They are very closely related:

- A **thread pool** is a general concept: reusable worker threads that run submitted tasks.
- A **Scheduler** is Reactor's wrapper around that concept, with a reactive-friendly API.

A Scheduler *owns* a thread pool (or a similar structure). It is the reactive-programming way of saying "here is a pool of threads you can submit work to."

```
Scheduler (Reactor concept)
    | owns
    ▼
Thread pool (general concept)
    | contains
    ▼
Worker threads (actual JVM threads)
    | execute
    ▼
Tasks (Runnable / lambdas)
```

Relationship in one diagram

```
Reactive pipeline  ── subscribeOn / publishOn / schedule ──►  Scheduler
                                                                | owns
Reactive pipeline ◄── run tasks on scheduler threads ──────────┘
                                                                ▼
                                                          Worker threads
```

---

9. Common Reactor Schedulers

Brief overview. (In modern Reactor versions, `Schedulers.immediate()` still exists but is rarely used — everything runs on the calling thread with no actual scheduling.)

Scheduler	Intended use	Type of work	Suitable for blocking?

`Schedulers.immediate()`	"No scheduling" — run on the current thread, right now	Any tiny quick work	No
`Schedulers.single()`	A single, dedicated thread	Small tasks that must share one thread (e.g., a scheduler used across a small app)	No (it is one thread; blocking it stalls everything on it)
`Schedulers.parallel()`	A fixed pool sized to the number of CPU cores	CPU-heavy computation	**No** (do not block CPU-bound threads)
`Schedulers.boundedElastic()`	A *bounded*, *elastic* pool	Potentially blocking work: DB calls, file I/O, HTTP calls, MinIO	**Yes — this is its purpose**

`Schedulers.immediate()`

- Runs the task on the **calling thread** immediately.
- It is basically "do nothing special."
- Not useful for moving work off a thread.

`Schedulers.single()`

- One single worker thread reused for all tasks.
- Good for shared background work that must be serialized.
- Since there is only one thread, blocking it is bad.

`Schedulers.parallel()`

- Sized to the number of CPU cores (fixed size).
- Intended for CPU-bound computation.
- Blocking work here would stall CPU work — **do not** use it for blocking calls.

`Schedulers.boundedElastic()`

- The recommended scheduler for **potentially blocking** work.
- Creates threads on demand (elastic) but with a hard **bound** (Section 10).
- This is the scheduler used in your Spring AI example.

> **Rule of thumb:** blocking / I/O-heavy work → `boundedElastic()`; CPU-heavy computation → `parallel()`.

---

10. `Schedulers.boundedElastic()`

The most important section

```
Schedulers.boundedElastic()
```

In very simple language: it is a pool of worker threads that can grow as needed, but **never beyond a hard limit**. It is designed specifically for work that may block (database calls, file I/O, MinIO, external HTTP).

Breaking down the name

`bounded` = limited. The pool cannot create an unlimited number of threads. There is a ceiling (default: 10 × number of CPU cores, capped at a maximum, per JVM).

`elastic` = stretchy. Threads are created on demand when there is more work, and idle threads can be reused or released later.

So: *"a thread pool that stretches when needed, but is still bounded by a limit."*

Why it is designed for potentially blocking work

Because the whole point of the *other* schedulers is to avoid blocking:

- `parallel()` threads are for CPU work — blocking them wastes CPU capacity.
- `single()` is a single thread — blocking it stalls everything.

Blocking work needs *more* threads than CPU work, because each blocked thread is idle-waiting. `boundedElastic` trades more (potentially idle) threads for the ability to run blocking code without freezing the reactive pipeline.

Worker threads and thread reuse

Like any pool, the worker threads are reused:

- A worker finishes task 1 → picks up task 2 → finishes → task 3...
- Threads are not destroyed after every task.

Limited thread creation

Even though it is "elastic," the pool will never go above its bound. If the limit is, say, 100 threads and they are all busy:

- The 101st task cannot get a thread.
- It **waits in a queue** until a thread becomes free.

Tasks waiting when workers are busy

```
1000 tasks
     ↓
boundedElastic scheduler
     ↓
limited number of workers   (e.g., 100)
     ↓
some tasks execute          (100 executing)
remaining tasks wait        (900 queued)
     ↓
worker becomes free
     ↓
next task executes
```

Why it is safer than creating unlimited threads

Unlimited threads = unlimited memory + OS overhead + huge context-switching cost → the app can crash or thrash. `boundedElastic()` prevents that by enforcing a cap, so the worst case is bounded and predictable.

Why it is NOT a solution to unlimited traffic

`boundedElastic()` protects *your JVM's threads*. It does **not**:

- Make blocking operations faster

- Remove the need for connection pools
- Protect MinIO, PostgreSQL, or OpenRouter from overload
- Make 10,000 concurrent blocking tasks succeed — many will simply **wait**

It bounds your threads; it does not bound your traffic.

---

11. Shared Scheduler Concept

The same JVM shares one scheduler

When you call the standard Reactor `Schedulers.boundedElastic()` from multiple places, you normally get **one shared bounded-elastic scheduler per JVM** — not a brand-new pool for every service.

In Spring Boot:

```
Spring Boot JVM
│
├── AI Service
│       └── boundedElastic()      ─┐
│                                  │  all point to the SAME shared
├── File Service                   │  bounded-elastic scheduler (default)
│       └── boundedElastic()      ─┤
│                                  │
└── Report Service                 │
        └── boundedElastic()      ─┘
```

(You can create separate pools with `Schedulers.newBoundedElastic(...)`, but calling the standard method gives you the shared one.)

Tasks compete for the scheduler's workers

Because they share one pool, tasks from the AI service, file service, and report service **compete for the same finite number of workers**.

```
Shared boundedElastic scheduler (e.g., 100 workers)
  AI task          ┐
  File task        ├──►  100 workers: some execute, the rest queue
  Report task      ┘
```

One busy service can consume many workers, making other services' tasks wait longer.

Horizontally scaled: each JVM is independent

When you scale out, each JVM has its **own** scheduler and its own threads.

```
Load Balancer
   ├── JVM 1 → Scheduler (own threads)
   ├── JVM 2 → Scheduler (own threads)
   └── JVM 3 → Scheduler (own threads)
```

- Thread resources are **per JVM**.
- Scaling out gives you *more* schedulers in total — but each JVM still has its own bound.
- This also means total queueing capacity scales with the number of instances.

---

12. What Happens with 1000 Requests?

Production scenario: 1000 requests arrive at your Spring Boot app.

1000 requests ≠ 1000 threads

This is the central idea:

> **1000 requests does NOT automatically mean 1000 threads.**

The requests become **tasks** that wait for available workers.

```
1000 requests
      ↓  (become tasks)
boundedElastic scheduler (e.g., 100 workers)
      ↓
100 tasks → executing
900 tasks → waiting in the queue
      ↓ (as workers free up)
next tasks execute
```

Thread pools have finite capacity

Whatever the pool bound is, that's the most concurrent blocking work in flight. Everything beyond the bound waits.

The queue helps, but large traffic still overwhelms

Even with a bounded thread pool, if tasks arrive faster than they complete:

- The queue grows.
- Memory grows.
- Downstream systems receive bursts of requests.

`boundedElastic()` does not make unlimited work possible

It only caps your threads. It can't cap MinIO load, database load, or memory.

All these can become bottlenecks

- **MinIO** — too many concurrent object operations at once
- **PostgreSQL** — too many connections/active queries (connection pool limits!)
- **OpenRouter** — external API rate limits and latency
- **Connection pools** — database connection pool is also finite
- **Memory** — 1000 buffered responses, queues, and buffers
- **CPU** — even at "1000 requests," parsing/processing work consumes CPU
- **Network** — bandwidth and latency

```
Queue/worker picture:

Requests ──► [ queue ] ──► [ workers: X ] ──► downstream (MinIO,
Postgres, OpenRouter)
                              │
                              └─► X workers are the ONLY concurrency
limit on the blocking side
```

Lesson: to handle 1000 requests well you need to think about the whole chain — thread pools, connection pools, downstream capacity, memory — not just threads.

---

13. `schedule()` Method

The exact code

```
Schedulers.boundedElastic().schedule(() -> {
    parseAndSaveFiles(fullResponseBuffer.toString(), projectId);
});
```

What each part does

`Schedulers.boundedElastic()` returns a `Scheduler` (specifically, the shared bounded-elastic scheduler of the JVM). It has worker threads ready to run tasks.

`.schedule(...)` submits the given task to that scheduler and **returns immediately** (asynchronously). It does not run the task on the current thread.

The lambda `() -> {...}` is the task — a `Runnable`. It is the piece of work (calling `parseAndSaveFiles(...)`) that will be executed by one of the scheduler's worker threads.

Why the task runs asynchronously

Because `schedule()` puts the task on the scheduler's queue/workers and returns at once:

- The calling code (the `doOnComplete` callback) continues without waiting.
- The blocking work (`parseAndSaveFiles`) runs later, on a worker thread.

```
current thread:   doOnComplete → schedule() → continue, NOT blocked
                          |
                          ▼ (asynchronously)
boundedElastic worker:    parseAndSaveFiles() → MinIO → PostgreSQL →
done
```

Why this is useful after an AI streaming response completes

`doOnComplete` fires when the AI stream finishes. At that moment you want to save the generated files — but that involves blocking I/O (MinIO, database). If you ran it directly on the reactive stream's thread, you would **block the streaming pipeline's thread**, hurting other streams.

Scheduling it onto `boundedElastic()` moves the blocking work off the reactive threads.

Plain English translation

> "When the stream completes, submit the potentially blocking file-processing work to a worker managed by the bounded-elastic scheduler."

---

14. Apply It to the Lovable Clone

The application context

```
React
   ↓
Spring Boot
   ↓
Spring AI
   ↓
OpenRouter
   ↓
LLM
```

The Lovable Clone:

1. User asks for an app in React.

2. Spring Boot forwards the prompt to Spring AI.
3. Spring AI calls OpenRouter → LLM.
4. The LLM response is **streamed** back to React, chunk by chunk.
5. After streaming finishes, the generated files must be **parsed and saved** to MinIO (storage) and PostgreSQL (metadata).

After streaming completes

```
AI stream completes
       ↓
parseAndSaveFiles()
       ↓
Parse generated files
       ↓
MinIO
       ↓
PostgreSQL
```

Why `parseAndSaveFiles()` belongs on `boundedElastic()`

`parseAndSaveFiles()` performs potentially blocking operations:

- **MinIO** → network + HTTP to an object store
- **PostgreSQL** → network + SQL round-trips
- **File I/O** → reading/writing content

Those wait for external systems, so they block threads. Running them on the reactive stream's thread would freeze the stream. Moving them to `boundedElastic()` lets a worker thread do the waiting while the reactive pipeline stays free.

The full picture



---

15. Explain the Exact Code

```
@Override
@PreAuthorize("@security.canEditProject(#projectId)")
public Flux<String> streamResponse(String userMessage, Long projectId) {
    Long userId = authUtil.getCurrentUserId();
    createChatSessionIfNotExists(projectId, userId);

    Map<String, Object> advisorParams = Map.of(
            "userId", userId,
            "projectId", projectId
    );

    StringBuilder fullResponseBuffer = new StringBuilder();

    return chatClient.prompt()
            .system("SYSTEM PROMPT HERE")
            .user(userMessage)
            .advisors(
                    advisorSpec -> {
                        advisorSpec.params(advisorParams);
                    }
            )
            .stream()
            .chatResponse()
            .doOnNext(response -> {
                String content =
response.getResult().getOutput().getText();
                fullResponseBuffer.append(content);
            })
            .doOnComplete(() -> {
                Schedulers.boundedElastic().schedule(() -> {
                    parseAndSaveFiles(fullResponseBuffer.toString(),
projectId);
                });
            })
            .doOnError(error ->
                    log.error(
                            "Error during streaming for projectId: {}",
```

```
                        projectId,
                        error
                )
        )
        .map(response ->
                Objects.requireNonNull(
                        response.getResult().getOutput().getText()
                )
        );
}
```

Where the reactive stream is created

`chatClient.prompt()...stream().chatResponse()` builds the reactive stream. `chatClient` is the Spring AI client; `.stream()` tells Spring AI you want the response streamed; `.chatResponse()` produces a `Flux<ChatResponse>`.

What `.stream()` does conceptually

It turns the LLM request into a **reactive stream**: as the model generates tokens, each chunk flows down the pipeline as an event. The client no longer waits for the whole answer before doing anything — it reacts to chunks as they arrive.

What `Flux<String>` represents

The method returns `Flux<String>`: a stream that emits **many `String` chunks** (the tokens/text of the AI answer) as they arrive, then completes. React will subscribe and display each chunk as it comes.

What `doOnNext()` is doing

A "side effect" hook that runs for **every emitted item** without changing the stream. Here it:

- Takes the current chunk of AI text
- Appends it to `fullResponseBuffer`

So `fullResponseBuffer` accumulates the whole answer, chunk by chunk, across the whole stream.

Why `StringBuilder` is used

`StringBuilder` is the efficient way to accumulate many string pieces. If you used `String + String` repeatedly, you would create many throwaway objects. `StringBuilder` appends in place — and because only the reactive stream's code writes to it here, it is safe to use a single instance for the lifetime of one stream.

What `doOnComplete()` means

A hook that runs **once, when the stream finishes normally** — after the last chunk was emitted. At that point the full AI answer is in `fullResponseBuffer`, so it is the right moment to do "after-stream" work like saving files.

Why `boundedElastic()` is used

Because `parseAndSaveFiles` does potentially blocking work (MinIO + PostgreSQL). `boundedElastic()` provides worker threads meant for that, keeping the blocking work off the reactive pipeline's threads.

What `schedule()` does

It submits `parseAndSaveFiles(...)` as an asynchronous task to the bounded-elastic scheduler and returns immediately. The reactive flow is not blocked waiting for the file saving.

Why `parseAndSaveFiles()` is executed separately

So the stream can complete and the HTTP response can finish cleanly, while the file saving happens in the background on a worker thread. The user gets their streamed answer without waiting for MinIO/DB work to finish on the same thread.

What happens if 1000 requests arrive

- 1000 streams run concurrently.
- Each request adds **one** file-saving task after its stream completes.
- All those tasks go to the **shared** bounded-elastic scheduler.
- Only a limited number execute at once; the rest **wait in the queue**.

What happens if MinIO is slow

- Worker threads doing MinIO calls stay occupied (blocked) longer.
- Fewer workers become free → queued file-saving tasks wait longer.
- If MinIO is very slow, the queue backs up and downstream latency grows.

What happens if `parseAndSaveFiles()` throws an exception

Here it is important:

- The task runs on the scheduler, **not** inside the reactive stream.
- An exception in a `Runnable` submitted via `schedule()` does **not** flow through the pipeline's `doOnError`.

The scheduled task is not part of the original `Flux` error propagation

The reactive stream and the scheduled task are **separate execution contexts**:

```
Flux pipeline  ── doOnError handles stream errors (from chatClient, map,
etc.)

separately scheduled task:  parseAndSaveFiles()
        |
        ▼
  runs on boundedElastic worker
        |
        ▼
  exceptions here do NOT reach the Flux's doOnError
```

Errors inside the scheduled task should be handled separately

If this design is kept, `parseAndSaveFiles` should catch and log/handle its own exceptions:

```
.doOnComplete(() -> {
    Schedulers.boundedElastic().schedule(() -> {
        try {
            parseAndSaveFiles(fullResponseBuffer.toString(), projectId);
        } catch (Exception e) {
            log.error("Failed to parse and save files for projectId:
{}", projectId, e);
        }
    });
})
```

Otherwise a failure inside the scheduled task could be silently lost.

---

16. Important Distinctions

Concept	What it is	Simple analogy
Thread	A path of execution that runs your code	One worker
Thread Pool	A reusable, limited set of threads	A team of workers
Task	A unit of work submitted to be executed	A job handed to a worker
Scheduler	Reactor's abstraction for "where work executes"	A dispatcher that assigns jobs to workers
Mono	A stream that emits 0 or 1 item	A promise of a single value
Flux	A stream that emits 0 to N items	A stream of many values
Synchronous	You wait for the operation to finish before continuing	Ordering a meal and standing at the counter until it's ready
Asynchronous	You start the operation and continue; the result arrives later	Ordering a meal, taking a number, getting called when ready
Blocking	The thread stops and waits for I/O	A worker waiting with nothing to do
Non-blocking	The thread does not wait; it's notified when data arrives	A worker doing other tasks until the notification arrives
Concurrency	Dealing with many things at once (interleaved)	One chef preparing several dishes by switching between them
Parallelism	Doing many things at the same time (real simultaneous execution)	Multiple chefs cooking at the same time on multiple stoves

---

17. Common Misconceptions

Misconception	The truth
"1000 requests means 1000 threads"	1000 requests become **tasks**; only a limited number of **workers** execute at once. The rest wait.
"`Flux` means multiple threads"	No. A `Flux` is just a stream of values. By default it runs on the subscribing thread. Threads are controlled by **Schedulers**.
"Asynchronous means a new thread is always created"	Not necessarily. Asynchronous just means "don't wait." Work can be handled by an event loop, a shared pool, callbacks, etc. — often without creating a new thread.
"`boundedElastic()` makes blocking operations non-blocking"	No. The operation is still blocking — it's just moved to a different thread. The *reactive pipeline* stays unblocked, but a worker thread still waits.
"More threads always means better performance"	No. Too many threads waste memory and cause context-switching overhead. Threads are often idle-waiting anyway.
"A thread pool has unlimited threads"	No. Pools have a **bound**. Extra tasks wait in a queue.
"Every service gets its own boundedElastic pool"	Not automatically. The standard `Schedulers.boundedElastic()` returns the **shared** JVM-wide scheduler.
"Using boundedElastic means the application cannot be overloaded"	No. It bounds *threads*, but MinIO, PostgreSQL, OpenRouter, memory, CPU, network, and connection pools can all still be overwhelmed.

---

18. Practical Rules to Remember

```
1. Threads execute code.
2. Thread pools reuse worker threads.
3. Tasks are not the same thing as threads.
4. Blocking operations occupy threads while waiting.
5. Schedulers control where reactive work executes.
6. boundedElastic is intended for potentially blocking work.
7. boundedElastic is bounded; it does not provide unlimited threads.
8. Many tasks can wait for a limited number of workers.
9. High traffic can still overload downstream services.
10. Use reactive/non-blocking APIs where possible.
```

Quick Revision Sheet

The story in 10 lines:

1. A thread is a path of execution through your code.
2. One thread = one thing at a time; apps use many threads to serve many requests.
3. Blocking operations (DB, file, MinIO, HTTP) make a thread sit and wait.
4. A thread pool reuses a limited set of threads; tasks wait for free workers.
5. Concurrency = many things happening (interleaved); parallelism = many things truly at the same time (needs cores).
6. Asynchronous means: start the work, don't wait; react when it finishes.
7. Reactor gives us `Mono` (0–1 value) and `Flux` (0–N values) streams; a `Flux` alone does not create threads.
8. A Scheduler is Reactor's thread-pool abstraction; it decides where tasks run.
9. `boundedElastic()` = a thread pool that grows on demand but is capped, meant for blocking work. It is shared per JVM.
10. When the AI stream completes, `schedule()` moves `parseAndSaveFiles()` (MinIO + PostgreSQL) onto a bounded-elastic worker, keeping the reactive stream free — but that task's errors must be handled separately.